

Introduction

Every developer starts the same way: something breaks, and the first instinct is to sprinkle `print()` statements around the code until the problem reveals itself. It works, sort of — right up until the code leaves your laptop. The moment a bug shows up in production, on a server nobody is watching, `print()` stops being a debugging tool and starts being noise you have to clean up later, if you remember to.

This matters more than it seems for a simple reason: debugging is not a peripheral skill you pick up eventually. It's the actual job. Writing code is maybe half of what a software engineer does; figuring out why code isn't doing what you expect is the other half, and it's the half interviewers quietly evaluate even when they're not asking a "debug this" question directly. Someone who can only debug by guessing and re-running is slower at everything — shipping features, reviewing PRs, responding to incidents.

The good news is that Python already ships with two tools that solve this properly, and neither requires installing anything. `pdb` (and its shortcut, `breakpoint()`) lets you freeze code mid-execution and inspect it like a live crime scene. The `logging` module gives you a print statement that never has to be deleted, because you can just turn it off. Combined with a simple strategy for narrowing down where a bug lives, these two tools turn a twenty-minute guessing game into a two-minute lookup.

Problem Overview

Here's the scenario in plain terms: you have a function that's supposed to calculate a discount for a user's cart, but it keeps returning `None` instead of a number. The function itself looks fine when you read it. The logic seems correct. So where's the bug?

The traditional approach is to add a `print()` statement above the suspicious line, run the code, look at the output, and repeat — moving the print statement around until something looks wrong. This works for genuinely trivial bugs. It falls apart the moment:

- The bug depends on live state you can't easily reconstruct (a running server, a real user session, a database in a particular state).
- The bug isn't in the function you're staring at, but in data that arrived *before* that function was ever called.

- You need to inspect more than one variable, or inspect them conditionally, or step through several function calls in sequence.

In our example, the bug isn't actually inside `get_discount` at all. It's upstream — bad data is arriving at the function's doorstep. No amount of print statements inside `get_discount` would tell you that as directly as pausing execution and looking at the incoming argument.

Example

```
def get_discount(user: dict) -> float:
    total = user["cart_total"]
    if total > 100:
        return total * 0.9
    return total * 0.95

cart = {"cart_total": None}
print(get_discount(cart))
```

Run this, and Python throws a `TypeError: '>' not supported between instances of 'NoneType' and 'int'`. The traceback points at the comparison inside `get_discount`, which makes it tempting to assume the bug lives there. It doesn't. The function is doing exactly what it's told — the problem is that `cart_total` was `None` before it ever reached this function. Something upstream (a failed API call, a missing form field, a bad default) produced bad data, and `get_discount` is just the first place it caused a visible crash.

This is the core insight the whole video builds toward: the line where code fails and the line where the bug originates are often two different places, and print statements are bad at telling them apart.

Intuition

An experienced engineer doesn't reach for more print statements when a bug is confusing — they reach for a way to *stop time*. The real question a debugger answers isn't "what does my code do," it's "what does my program actually look like, right now, at this exact line, with these exact values." Print statements approximate that by showing you fragments after the fact. A debugger gives you the whole picture, live, and lets you ask follow-up questions on the spot instead of guessing what the next print statement should say.

The second piece of intuition is about *where* to look. Instead of inspecting every line, an experienced engineer bisects the problem: pick a line roughly in the middle of where the bug could be, check whether the bad state exists before or after it, and repeat. This is the same idea as binary search,

applied to a bug instead of a sorted array — each check eliminates half the remaining code, so you converge on the exact line in a handful of steps instead of scanning the whole file top to bottom.

Once you accept that debugging is really "pause, inspect, narrow" repeated a few times, the specific tools become obvious. You need something that pauses (`breakpoint()`), something that records without needing to be there live (`logging`), and a strategy for choosing where to pause (bisection).

Brute Force Solution

The brute-force approach is scattering `print()` calls:

```
def get_discount(user: dict) -> float:
    print("DEBUG: user =", user)
    total = user["cart_total"]
    print("DEBUG: total =", total)
    if total > 100:
        return total * 0.9
    return total * 0.95
```

Idea: insert output statements near anything suspicious, run the program, read the console.

Algorithm: guess a location → add a print → run → read output → if not helpful, add another print elsewhere → repeat.

Advantages: - Zero setup, works everywhere immediately. - Easy for a beginner to understand.

Disadvantages: - Doesn't work once the code is running somewhere you can't watch it live (a server, a scheduled job, a container). - Every print statement has to be manually added and later manually removed — forget one and it ships to production. - Only shows you what you thought to print. If the actual bug is in a variable you didn't think to inspect, you get nothing. - No severity levels — you either see everything or you comment lines out one by one.

Time complexity: effectively $O(n)$ *edit-run cycles*, where n is the number of guesses before you get lucky — unbounded in the worst case, since a wrong guess costs a full round trip.

Space complexity: N/A — this is a workflow cost, not a memory cost, but the "space" it costs is your own time and attention.

Optimal Solution

The optimal approach swaps guessing for pausing, and swaps manual cleanup for a dial you can turn.

Step 1 — Pause execution with `breakpoint()`. Drop `breakpoint()` on the line right before you suspect trouble. When Python reaches it, execution freezes and drops you into a `pdb` prompt — an interactive shell sitting inside your running program.

Step 2 — Use five commands to look around.

Command	What it does
<code>p <expr></code>	Print the value of a variable or expression
<code>n</code>	Step to the next line (don't go inside function calls)
<code>s</code>	Step into a function call
<code>c</code>	Continue running until the next breakpoint (or the end)
<code>l</code>	List the surrounding source code

`n` and `p` alone resolve the majority of bugs — step forward, print whatever looks suspicious, repeat. `s`, `c`, and `l` are there for when you need to go deeper.

Step 3 — Once you find it, switch to `logging` for anything that needs to survive outside your terminal.

```
import logging
logging.basicConfig(level=logging.WARNING)
```

Every log call carries a timestamp and a severity level automatically — `debug`, `info`, `warning`, `error`. The `level` argument acts as a floor: anything below it is silently dropped, without touching the calls themselves. Set the floor to `WARNING` and your `debug` / `info` calls stay in the code but produce no output. Need more detail during an incident? Change one argument to `DEBUG`, and every line reappears — no code changes, no redeploy just to see more.

Step 4 — Bisect instead of scanning. Rather than debugging the whole file, ask one question repeatedly: does the bug exist *before* this line, or *after* it? Each answer eliminates half the remaining code, so a file with dozens of candidate lines collapses to the right one in just a few pauses.

Python Code

```
import logging

logging.basicConfig(level=logging.WARNING)

def get_discount(user: dict) -> float:
    """Return the discounted cart total for a user."""
    breakpoint() # pause here to inspect `user` before `total` is used
```

```
total = user["cart_total"]
logging.debug("cart_total resolved to %s", total)

if total is None:
    logging.error("cart_total was None for user=%s", user)
    raise ValueError("cart_total is missing or None")

if total > 100:
    return total * 0.9
return total * 0.95

if __name__ == "__main__":
    cart = {"cart_total": None}
    print(get_discount(cart))
```

Code Walkthrough

- `logging.basicConfig(level=logging.WARNING)` sets the global floor for log output. Only `WARNING` and `ERROR` calls print; `DEBUG` and `INFO` calls still execute but produce nothing.
- `breakpoint()` freezes execution at that exact line, before `total` is even assigned, and drops into the `pdb` prompt.
- Inside the prompt, `p user` prints the entire dictionary — this is where you'd see `cart_total` is `None` immediately, without stepping any further.
- `logging.debug(...)` is a permanent, harmless line. It costs nothing when the floor is `WARNING` and becomes useful the instant you need to raise the floor to `DEBUG`.
- The explicit `if total is None` guard converts a confusing downstream `TypeError` into a clear, immediate `ValueError` with context about which user triggered it — this is what production code should do instead of crashing three lines later with no explanation.
- `logging.error(...)` records the failure with the actual bad state attached, so even if nobody was watching a terminal when it happened, the log file tells the story afterward.

Dry Run

Using `cart = {"cart_total": None}`:

1. `get_discount(cart)` is called.
2. Execution hits `breakpoint()` and pauses before `total` is assigned.
3. You type `p user` → output: `{'cart_total': None}`. The bad data is visible immediately.
4. You type `c` to continue.
5. `total = user["cart_total"]` assigns `None`.

6. `logging.debug(...)` runs but produces no output (floor is `WARNING`).
7. `if total is None: is True` → `logging.error(...)` prints a timestamped error line, then `ValueError` is raised.

Total elapsed debugging time: one breakpoint, one `p` command. Compare that to the opening scenario, where the same bug took twenty minutes of relocating print statements.

Complexity Analysis

This isn't an algorithm with input size `n`, but the same reasoning applies to *debugging effort*:

- **Brute force (print debugging):** effort scales roughly linearly with how many wrong guesses you make before finding the right line — unbounded, since nothing narrows the search space systematically.
- **Optimal (breakpoint + bisection):** effort scales with $O(\log n)$ pauses, where `n` is the number of candidate lines, because each bisection step eliminates half the remaining code. That's the same reason binary search beats a linear scan — you're not checking every line, you're halving the search space each time.

Space cost is unaffected in both cases; this is purely about the number of round trips needed to locate the bug.

Alternative Solutions

- **IDE breakpoints (VS Code, PyCharm):** functionally similar to `pdb` — pause, inspect, step — but tied to a specific editor and its UI. `breakpoint()` is text-based, works over SSH, in Docker containers, and in any terminal, which makes it more portable for backend and server-side debugging.
- **logging with structured JSON output:** for larger production systems, plain-text log lines are often replaced with structured logs (JSON) so they can be queried by tools like Elasticsearch or Datadog. The mental model is identical — severity levels and a filterable floor — just with a different output format.
- **pdb++ / ipdb :** drop-in replacements for `pdb` with syntax highlighting and tab completion. Worth adopting once you're comfortable with the stock five commands; not necessary to get started.

Edge Cases

- **Empty input:** `get_discount({})` raises a `KeyError` before `breakpoint()` even matters — worth guarding with `user.get("cart_total")` if missing keys are expected.
- **`cart_total` is `None`:** handled explicitly above, converted from a confusing downstream error into a clear one.
- **`cart_total` is exactly `100`:** falls into the `else` branch (`> 100` is `False`), applying the 5% discount, not the 10% one — a common off-by-one-style boundary bug worth testing directly.
- **`cart_total` is negative:** the function will happily return a "discounted" negative number; whether that's valid depends on business rules the function doesn't currently enforce.
- **Breakpoint inside a loop running many iterations:** `breakpoint()` pauses on *every* iteration, which quickly becomes unusable for large loops. Use a conditional breakpoint (`if i == 500: breakpoint()`) or `pdb`'s conditional breakpoint support instead of pausing blindly on every pass.

Common Mistakes

1. **Leaving `breakpoint()` in code that gets deployed.** It will freeze a production process waiting for a terminal input that will never come. Always grep for stray `breakpoint()` calls before shipping.
2. **Forgetting to remove print statements**, which either ship to production or get accidentally committed as noise in a diff.
3. **Setting the logging floor to `DEBUG` in production long-term**, flooding log storage with noise that makes real errors harder to find.
4. **Debugging top-to-bottom instead of bisecting**, which turns a $O(\log n)$ search into something closer to $O(n)$.
5. **Assuming the crash location is the bug location.** As shown above, the line that raises the exception is often just the first place bad data becomes visible, not where it originated.
6. **Using `print()` for anything meant to persist.** Once the terminal closes, the output is gone; `logging` can be routed to a file or external service so it survives past the run.

Interview Questions

- "How would you debug a function that intermittently returns the wrong result in production, where you can't attach a live debugger?"
- "What's the difference between `logging.debug` and `print`, and when would you use each?"
- "Explain how you'd narrow down a bug in a 2,000-line file without reading every line."
- "What happens if you put `breakpoint()` inside a loop that runs 10,000 times?"
- "How do log levels let you control verbosity without redeploying code?"

Similar Problems

- **LeetCode-style debugging katas** (intentionally broken functions with subtle off-by-one or type errors) — good practice for applying `pdb` instead of reading code by eye.
- **Any "trace the output" question** (common in interview prep) — bisection and variable inspection are the same skill applied to a whiteboard instead of a terminal.
- **Exception-handling design problems** (e.g., "design a robust `parse_config` function") — the `is None` guard pattern shown here generalizes to any function that receives untrusted or externally sourced input.

Key Takeaways

- `print()` debugging doesn't scale past your own machine — it needs someone watching a terminal in real time.
- `breakpoint()` freezes execution and lets you inspect live state with five commands: `p`, `n`, `s`, `c`, `l`.
- `logging` is a print statement that never has to be deleted — control verbosity with one level argument instead of adding or removing code.
- Bisecting the problem (before this line or after?) turns a linear scan into a logarithmic one.
- The crash site and the bug's origin are often different places — inspect the data, not just the error message.

Watch the Video

This entire workflow — catching the live bug, flipping the logging level, and debugging the same problem twice to compare a guesswork approach against a systematic one — is demonstrated step by step in the video: {LINK}

About the Series

This lesson is part of *Fun with Learning Technology*, a series focused on making the parts of coding that usually get skipped over — tooling, debugging, the practical habits senior engineers rely on daily — actually make sense. Rather than just covering syntax, the series builds real workflows you can use the next time something breaks.

Call To Action

If this saved you from another twenty-minute print-statement hunt, the like button is the easiest way to say so. Subscribe on YouTube for the rest of the series, and subscribe to the Substack newsletter for deeper written breakdowns like this one. Drop a comment with your guess on what happens when a `breakpoint()` sits inside a loop running a thousand times — and share this with anyone still debugging exclusively with `print()`.

Quick Review

Print debugging: why avoid it for tricky bugs?

No state inspection, clutters code, easy to forget and ship. Trigger: bug needs repeated re-runs to isolate → use `breakpoint()` instead.

How do you pause execution mid-function?

Insert `breakpoint()` (or `import pdb; pdb.set_trace()`) at the line — drops you into `pdb` shell at that exact point.

Name `pdb`'s five essential commands.

`n` (next), `s` (step in), `c` (continue), `p <expr>` (print), `l` (list) — covers nearly all interactive debugging.

Why does `print()` fail in production debugging?

Stdout is often unmonitored/discarded in prod; `breakpoint()` can't attach to a live server either — logging module is the fix.

logging vs print for permanent diagnostics?

logging has levels (DEBUG/INFO/WARNING/ERROR), timestamps, and can be silenced via one config flag instead of deleting code.

Common bug when leaving debug logging in code?

Forgetting to flip the log level back up (e.g. left at DEBUG) floods production logs or hides real errors at wrong severity.

Python Lesson 30: Debugging and Logging (Using pdb/breakpoint(), the logging module, and systematic debugging strategies)

— free Python cheat sheet